

UCS3513 SYSTEM DESIGN LABORATORY

Lab Exercise 1 - Study of System Design Concepts and Development Tools

Name: Feminna G

Register Number: 3122245001704

Date: 06.07.2026

Aim

To study the fundamentals of system design, understand the components of scalable distributed systems, compare the tools used in the laboratory, and configure a Java project in Visual Studio Code for version control using GitHub.

System Design Concepts

System design is the process of defining an application's architecture, components, data storage and communication. Functional requirements describe what the application must do. Non-functional requirements describe qualities such as response time, availability, security and scalability.

Scalability: The ability to handle increasing users, requests or data. Vertical scaling increases the resources of one machine; horizontal scaling adds more machines.

Latency and Throughput: Latency is the time taken to complete a request. Throughput is the number of requests processed per unit time.

Availability and Reliability: Availability describes whether a service is accessible when required. Reliability describes its ability to operate correctly over time. Redundancy and recovery mechanisms help reduce the impact of failures.

Load Balancing: A load balancer distributes incoming requests among application servers. Health checks help avoid sending requests to unavailable servers.

Caching: A cache stores frequently accessed data for quick retrieval. It can reduce database work, but expiration and invalidation must be considered to prevent stale results.

Replication and Sharding: Replication maintains copies of data for availability and, where appropriate, read scaling. Sharding divides data across servers to distribute storage and workload.

Example of a Scalable Architecture

A client sends a request to a load balancer, which selects an application server. The server checks a cache before querying the database. A database replica can serve suitable reads, while a message queue can send time-consuming jobs to background workers. These components are introduced according to the workload; a small application may initially use a single server.

Comparison of Development Tools

Tool	Role in the laboratory	Typical verification / use
Java JDK	Compiles and runs Java programs.	java -version; javac -version
Visual Studio Code	Edits, runs and debugs source code with extensions.	Open the project and run Main.java.
Maven	Manages Java dependencies and build tasks.	mvn -version
Spring Boot	Builds Java web applications and REST APIs.	Start an application and call an endpoint.
Git	Tracks source changes and supports commits and branches.	git --version; git status
GitHub	Hosts Git repositories for backup and collaboration.	Check committed files in the repository.
Postman	Sends HTTP requests and inspects API responses.	Send GET/POST requests to a local API.
Redis	Stores in-memory data for caching and shared state.	redis-cli ping returns PONG when running.
MongoDB	Stores documents and supports replication and sharding.	Connect with mongosh and query a collection.
Apache Kafka	Provides event streaming between producers and consumers.	Publish and consume a test message.
Apache JMeter	Generates load and measures API performance.	Configure a Thread Group and HTTP sampler.
Docker / Compose	Runs services in containers and coordinates multiple services.	docker --version; docker compose version

Installation and Configuration Notes

Use a JDK for Java development and configure it in VS Code. Install the Extension Pack for Java for language support and debugging. Maven manages the dependencies needed by later Spring Boot experiments. Git must be configured with the correct author name and email before committing.

Redis, MongoDB, Kafka and JMeter are studied here and configured when required by the corresponding experiments. Listing a tool does not imply that it has been installed or tested for this record.

Procedure

1. Study the functional and non-functional requirements of a distributed application and identify the purpose of load balancing, caching, databases and background processing.
2. Compare the laboratory tools according to their development, storage, testing and deployment roles.
3. Verify the Java development environment using `java -version` and `javac -version`. Install a JDK if the compiler is unavailable.
4. Open the System Design laboratory folder in Visual Studio Code and enable Java support.
5. Create or reuse the Experiment-1 folder. Inside it, create `src/Main.java` containing the program shown below.
6. Run the program from the Experiment-1 folder and check the printed message.
7. Add a `.gitignore` file to exclude compiled class files and the `bin` directory from version control.
8. Use the existing laboratory Git repository if available. Check `git status` and `git remote -v` before making changes.
9. Stage only the Experiment-1 files, review the staged changes and commit them. Push to the configured GitHub repository using the existing branch.
10. Open the repository in a browser and verify that `Experiment-1/src/Main.java` is visible.

Code

src/Main.java

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("System Design Lab - Java Environment Ready");  
    }  
}
```

Run from the Experiment-1 folder:

```
java src/Main.java
```

Alternative compilation with a JDK:

```
mkdir -p bin  
javac -d bin src/Main.java  
java -cp bin Main
```

.gitignore

```
*.class  
bin/  
.DS_Store
```

Git Commands

For an existing repository, run the following from the repository root. Review the staged changes before committing. The push command assumes the current branch already has an upstream configured.

```
git status
git remote -v
git add Experiment-1/
git diff --cached
git commit -m "Complete Experiment 1 Java setup"
git push
```

If Git reports that there is nothing to commit, the project may already be saved. Check the GitHub folder instead of creating another repository. For a new repository, initialize Git, create a GitHub repository, configure its actual remote URL and push the initial commit after authentication.

Output

Verified Java execution

The source program was tested using OpenJDK 17. The captured text output is shown below.

```
$ java src/Main.java
System Design Lab - Java Environment Ready
```

Learning Outcomes

- Understand the basic requirements and components of scalable systems.
- Distinguish vertical scaling, horizontal scaling, replication and sharding.
- Identify the roles of development, storage, testing and deployment tools.
- Write and run a simple Java program.
- Understand the workflow for tracking Java source code with Git and publishing it on GitHub.

References

1. Faculty handout: Lab Exercise 1 - Study of System Design Concepts and Development Tools.
2. VS Code Java guide: <https://code.visualstudio.com/docs/java/java-tutorial>
3. GitHub Docs: Adding locally hosted code to GitHub.